

AI ASSISTED LAB ASSIGNMENT-2.1

Exploring Additional AI Coding tools beyond copilot

NAME: T. SATHVIK

Roll_No:2503A52415

Batch_No:14

Task-1: Statistical Summary For Survey Data.

INPUT:

#Generate A python function that reads a list of numbers and calculations the mean, minimum and maximum values.

```
def calculate_statistics(numbers):
```

```
    if not numbers:
```

```
        return None, None, None # Return None for mean, min, and max if the list is empty
```

```
    mean_value = sum(numbers) / len(numbers)
```

```
    min_value = min(numbers)
```

```
    max_value = max(numbers)
```

```
    return mean_value, min_value, max_value
```

```
# Example usage:
```

```
numbers = [10, 20, 30, 40, 50]
```

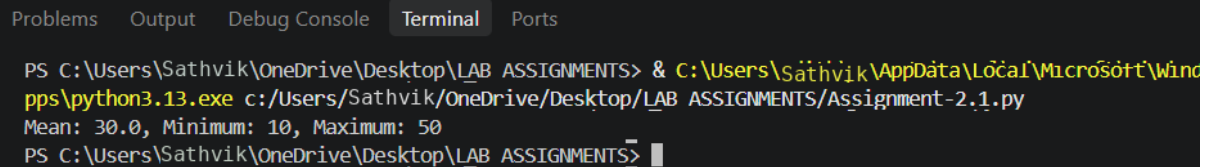
```
mean, minimum, maximum = calculate_statistics(numbers)
```

```
print(f"Mean: {mean}, Minimum: {minimum}, Maximum: {maximum}")
```

Explanation:

The `calculate_statistics()` function efficiently calculates the mean, minimum, and maximum of a list of numbers. It first checks whether the list is empty to avoid errors. Then it uses `sum()` and `len()` to calculate the mean and the built-in `min()` and `max()` functions to find the smallest and largest values.

OUTPUT:



```
Problems Output Debug Console Terminal Ports
PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS> & C:\Users\Sathvik\AppData\Local\Microsoft\Windows\apps\python3.13.exe c:/Users/Sathvik/OneDrive/Desktop/LAB ASSIGNMENTS/Assignment-2.1.py
Mean: 30.0, Minimum: 10, Maximum: 50
PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS>
```

Task-2: Armstrong Number – AI Comparison.

INPUT:

#Generate an Armstrong number checker using Given a number, check if it is an Armstrong number or not. Compare their outputs, logic style, and clarity.

```
def is_armstrong_number(n):
```

```
    # Convert the number to a string to easily iterate over each digit
```

```
    digits = str(n)
```

```
    num_digits = len(digits)
```

```
    # Calculate the sum of each digit raised to the power of the number of digits
```

```
    armstrong_sum = sum(int(digit) ** num_digits for digit in digits)
```

```
    # Check if the calculated sum is equal to the original number
```

```
    return armstrong_sum == n
```

```
# Example usage:
```

```
number = 153
```

```
if is_armstrong_number(number):
```

```
    print(f"{number} is an Armstrong number.")
```

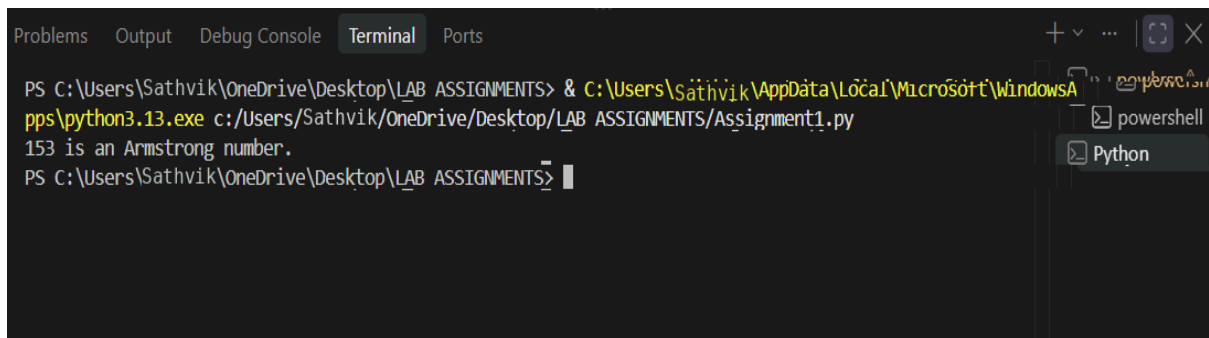
else:

```
print(f"{number} is not an Armstrong number.")
```

Explanation:

An Armstrong number is a number in which the sum of each digit raised to the power of the number of digits is equal to the original number.

OUTPUT:



```
PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS> & C:\Users\Sathvik\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:/Users/Sathvik/OneDrive/Desktop/LAB ASSIGNMENTS/Assignment1.py
153 is an Armstrong number.
PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS>
```

Task-3: Leap Year Validation Using Cursor AI.

INPUT:

#generate a Python program that checks whether a given year is a leap year.also give two different methods to check for leap year and compare their outputs, logic style, and clarity.

```
def is_leap_year_method1(year):
```

```
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
```

```
        return True
```

```
    else:
```

```
        return False
```

```
# Example usage:
```

```
year = 2020
```

```
if is_leap_year_method1(year):
```

```
    print(f"{year} is a leap year.")
```

```
else:
```

```
print(f"{year} is not a leap year.")
```

Method 2: Using a more concise approach

```
def is_leap_year_method2(year):
```

```
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

Example usage:

```
year = 2021
```

```
if is_leap_year_method2(year):
```

```
    print(f"{year} is a leap year.")
```

```
else:
```

```
    print(f"{year} is not a leap year.")
```

#example usage:

```
year = 2024
```

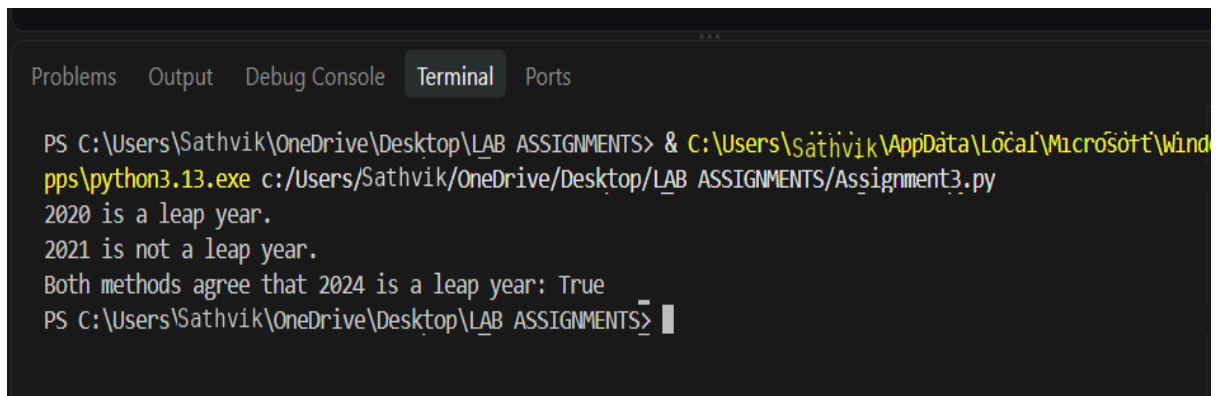
```
if is_leap_year_method1(year) == is_leap_year_method2(year):
```

```
    print(f"Both methods agree that {year} is a leap year: {is_leap_year_method1(year)}")
```

Explanation:

A leap year is a year that contains 366 days instead of 365. In Python, we can check whether a year is a leap year using divisibility rules. A year is a leap year if it is divisible by 4, but century years must also be divisible by 400. The program uses two methods to perform this check. The first method uses an if-else statement, while the second method uses a shorter Boolean expression. Both methods produce the same output. Method 1 is easier for beginners to understand, while Method 2 is more concise and clear for experienced programmers.

OUTPUT:



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS> & C:\Users\Sathvik\AppData\Local\Microsoft\Windows\apps\python3.13.exe c:/Users/Sathvik/OneDrive/Desktop/LAB ASSIGNMENTS/Assignment3.py
2020 is a leap year.
2021 is not a leap year.
Both methods agree that 2024 is a leap year: True
PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS> |
```

Task-4: Student Logic + AI Refactoring (Odd/Even Sum).

INPUT:

Write a python programme that calculates the sum of odd and even numbers in a tuple.

```
def sum_odd_even(numbers):
```

```
    odd_sum = sum(x for x in numbers if x % 2 != 0)
```

```
    even_sum = sum(x for x in numbers if x % 2 == 0)
```

```
    return odd_sum, even_sum
```

```
# Example usage:
```

```
tuple_numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
odd_total, even_total = sum_odd_even(tuple_numbers)
```

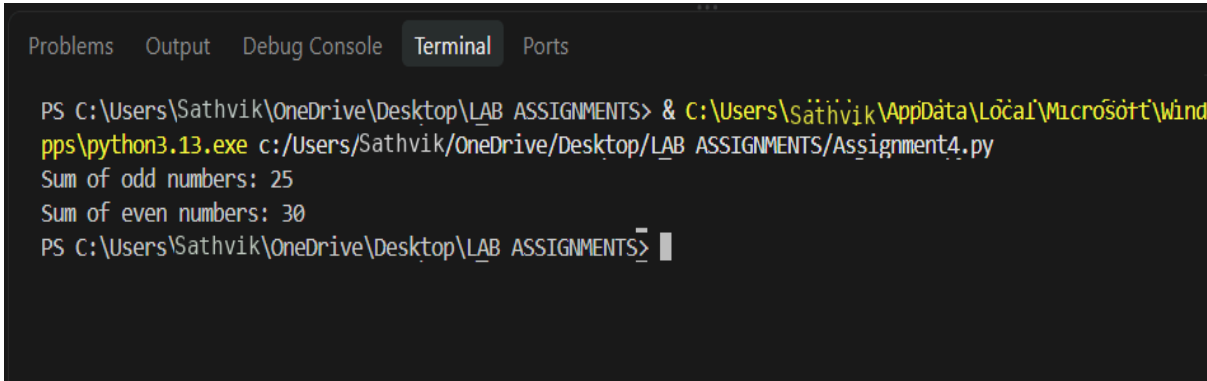
```
print(f"Sum of odd numbers: {odd_total}")
```

```
print(f"Sum of even numbers: {even_total}")
```

Explanation:

This Python program calculates the sum of odd and even numbers in a tuple. The function checks each number using the modulus (%) operator. If the remainder after division by 2 is not zero, the number is odd; otherwise, it is even. The sum() function adds the numbers separately. Finally, the program displays the total of odd numbers as 25 and even numbers as 30.

OUTPUT:



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS> & C:\Users\Sathvik\AppData\Local\Microsoft\Windows\
pps\python3.13.exe c:/Users/Sathvik/OneDrive/Desktop/LAB ASSIGNMENTS/Assignment4.py
Sum of odd numbers: 25
Sum of even numbers: 30
PS C:\Users\Sathvik\OneDrive\Desktop\LAB ASSIGNMENTS> █
```